

COMP90038
Algorithms and Complexity

Cezary Kaliszyk
(thanks to Harald Søndergaard)

Lecture 22

NP-Completeness

Concrete Complexity

Analysis of concrete algorithms

Their running times: best, average, worst case

Concrete Complexity

Analysis of concrete algorithms

Their running times: best, average, worst case

Approach

- give a bound for the asymptotic behaviour of running time
- as a function of input size

Examples

- quicksort: worst case $O(n^2)$
- mergesort: $O(n \log n)$



Abstract Complexity

Abstract Complexity

Complexity theory instead asks

"What is the inherent difficulty of the problem?"

Abstract Complexity

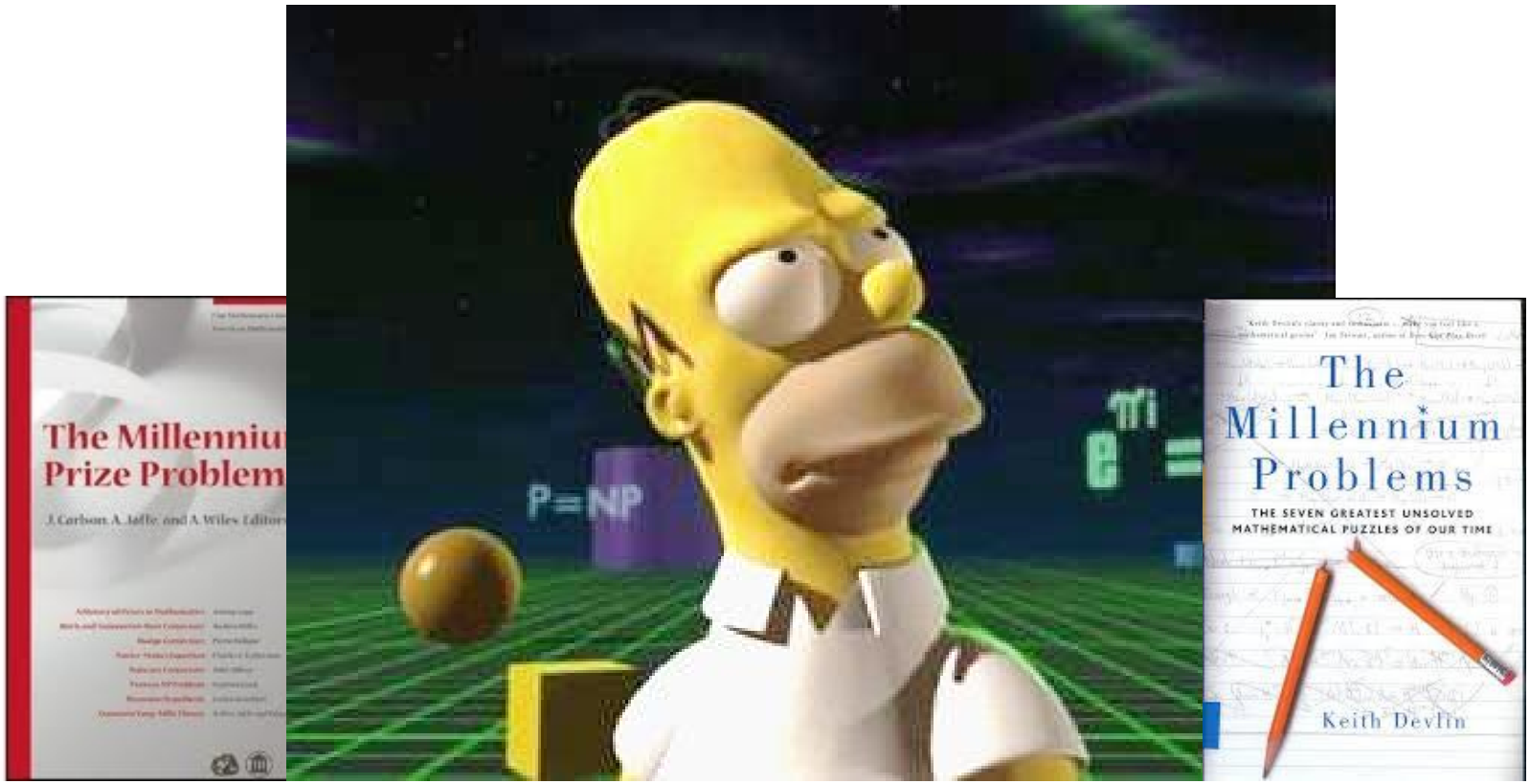
Complexity theory instead asks

"What is the inherent difficulty of the problem?"

- Is an algorithm optimal
- (in the asymptotic sense)

A Million Dollar Question: Is P = NP?

This is one of the seven “millennium problems”: The Clay Institute’s seven most important unsolved mathematical problems.



Algorithmic Problems

Problem in computer science

almost always means a family of instances of a general problem

Algorithmic Problems

Problem in computer science

almost always means a family of instances of a general problem

Algorithm for the problem

must work for all possible instances (input)

Algorithmic Problems

Problem in computer science

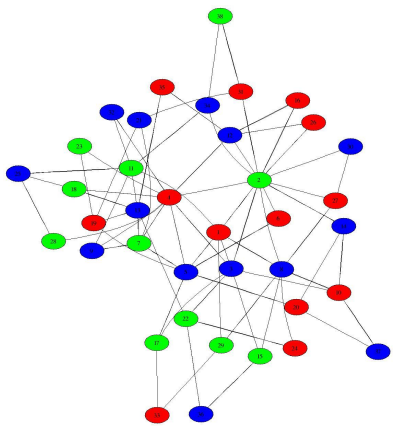
almost always means a family of instances of a general problem

Algorithm for the problem

must work for all possible instances (input)

Example: The sorting problem—an instance is a sequence of items.

Example: The graph k-colouring problem—an instance is a graph.



Example: Equation solving problems—an instance is a set of, say, linear equations.

Easy and Hard Problems

Definition: simple path

A path in a graph G is simple if it visits each node of G at most once.

Easy and Hard Problems

Definition: simple path

A path in a graph G is simple if it visits each node of G at most once.

Consider this problem for undirected graphs G:

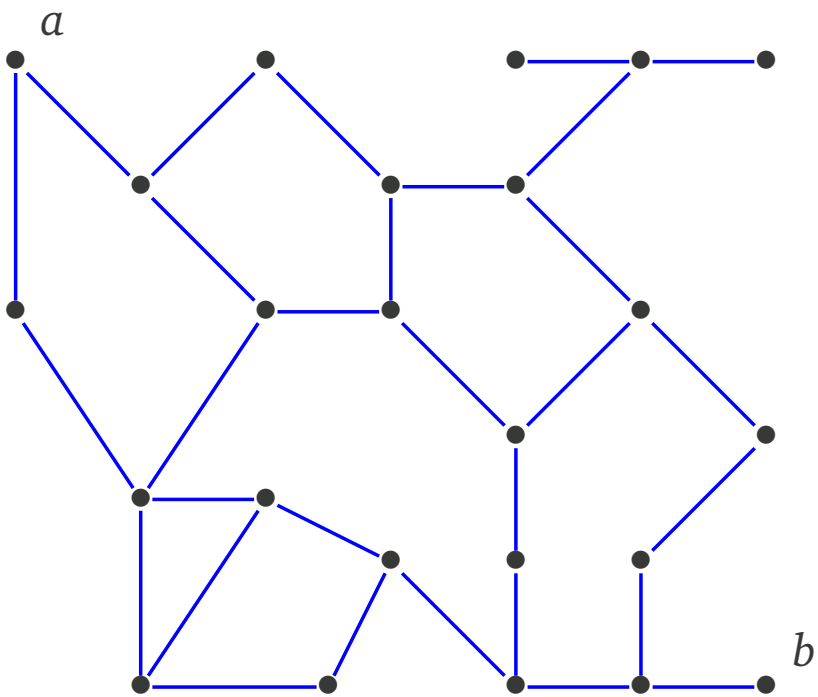
SPATH: Given G and two nodes a and b in G,
is there a simple path from a to b of length at most k?

And this problem:

LPATH: Given G and two nodes a and b in G,
is there a simple path from a to b of length at least k?

If you had a large graph G, which of the two problems would you rather have to solve?

Easy and Hard Problems



There are fast algorithms to solve SPATH.

Nobody knows of a fast algorithm for LPATH.

It is likely that the LPATH problem cannot be solved in polynomial time.

(But we don't know for sure.)

Eulerian Tours

The Eulerian tour problem is this:

> EUL: In a given graph, is there a path which visits each edge of the graph once, returning to the origin?

Is there a Eulerian tour of this graph?



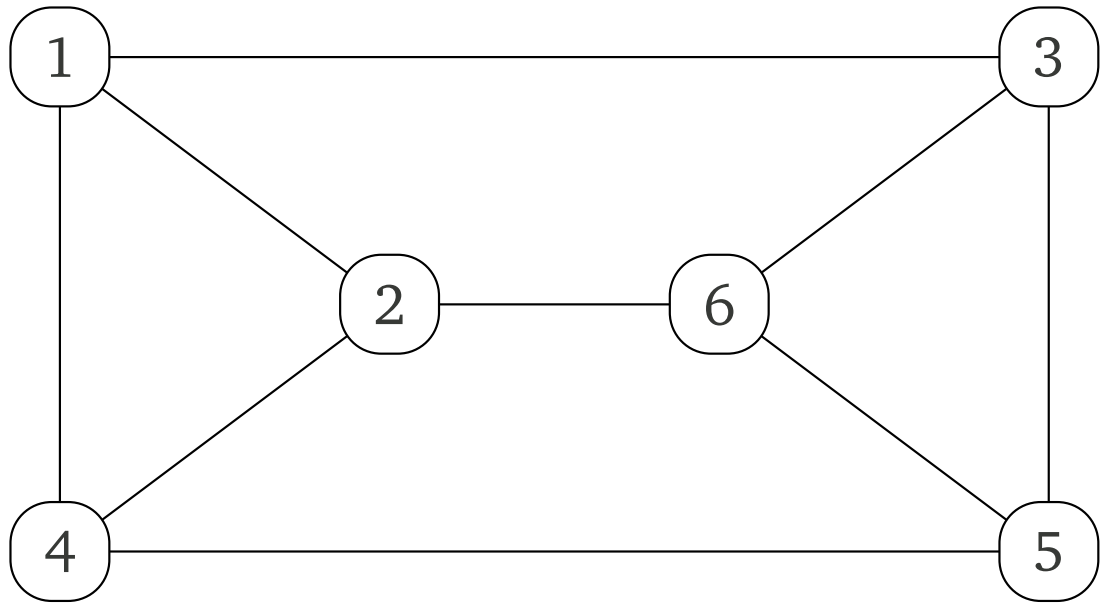
Eulerian Tours

The Eulerian tour problem is this:

EUL: In a given graph, is there a path which visits each edge of the graph once, returning to the origin?

Is there a Eulerian tour of this graph?



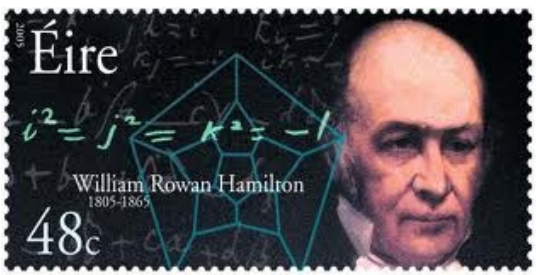


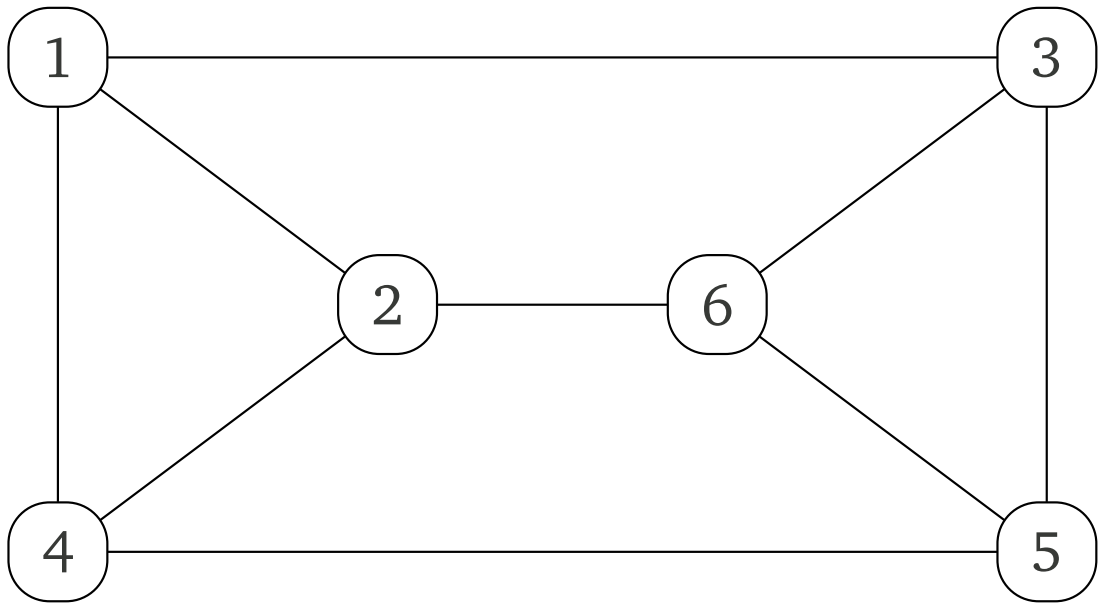
Hamiltonian Tours

The Hamiltonian tour problem again:

HAM: In a given graph, is there a path which visits each node of the graph once, returning to the origin?

Is there a Hamiltonian tour of this graph?





More Problems

Try to rank these problems according to inherent difficulty:

SAT

Given a propositional formula φ, is φ satisfiable?

More Problems

Try to rank these problems according to inherent difficulty:

SAT

Given a propositional formula φ, is φ satisfiable?

SUBSET-SUM

Given a set S of positive integers and a positive integer t, is there a subset of S that adds up to t?

More Problems

Try to rank these problems according to inherent difficulty:

SAT

Given a propositional formula φ, is φ satisfiable?

SUBSET-SUM

Given a set S of positive integers and a positive integer t, is there a subset of S that adds up to t?

3COL

Given a graph G, is it possible to colour the nodes of G using only three colours, so that no edge connects two nodes of the same colour?

Polynomial-Time Verifiability

If someone claims to have a solution

- For SAT, SUBSET-SUM, 3COL, LPATH, or HAM
- Testing the solution (a "yes instance") can be done quickly

Polynomial-Time Verifiability

If someone claims to have a solution

- For SAT, SUBSET-SUM, 3COL, LPATH, or HAM
- Testing the solution (a "yes instance") can be done quickly

Hard to solve

- But polynomial-time verifiable
- Many interesting problems in planning, scheduling, design, information retrieval, networks, games, ...

Deterministic vs. Non-deterministic algorithms

Deterministic vs. Non-deterministic algorithms

Given an instance I of a decision problem:

Stage 1: Guess an arbitrary string S as a candidate solution

Stage 2: A deterministic algorithm takes both I and S and outputs yes if S is a solution.

This non-deterministic algorithm solves a problem if it says yes for every yes instance.

P and NP

P is the class of decision problems solvable in polynomial time by a deterministic algorithm.

NP is the class of problems solvable in polynomial time by a non-deterministic polynomial algorithms.

Clearly $P \subseteq NP$. Is $P = NP$?

Most computer scientists consider this computer science's

most important open question

Problem Reduction

- The main tool used for classifying problems is reducibility
- Later we will be very precise about what we mean by reduction
- Simple cases of problem reduction:
  e.g. we translate some scheduling problem to the problem of topological sorting.

Two Very Similar Integer Problems

Difference constraints, DC:
Given a set of constraints of the form $x - y \in [k_1, k_2]$, find a solution or determine that no solution exists.

Two Very Similar Integer Problems

Difference constraints, DC:
Given a set of constraints of the form $x - y \in [k_1, k_2]$, find a solution or determine that no solution exists.

The set of constraints

$$\begin{aligned} x - y &\in [10, 11] \\ y - z &\in [7, 8] \\ x - z &\in [1, 4] \end{aligned}$$

has no solution in standard arithmetic.

Two Very Similar Integer Problems

Difference constraints, DC:
Given a set of constraints of the form $x - y \in [k_1, k_2]$, find a solution or determine that no solution exists.

Difference constraints modulo m, DC_m:
As DC, except variables and operations are those of modulo-m arithmetic.

We are interested in modulo-m arithmetic because that is what a computer implements ($m = 2^{32}$ or $m = 2^{64}$).

The set of constraints

$$\begin{aligned} x - y &\in [10, 11] \\ y - z &\in [7, 8] \\ x - z &\in [1, 4] \end{aligned}$$

has no solution in standard arithmetic.

Two Very Similar Integer Problems

Difference constraints, DC:
Given a set of constraints of the form $x - y \in [k_1, k_2]$, find a solution or determine that no solution exists.

Difference constraints modulo m, DC_m:
As DC, except variables and operations are those of modulo-m arithmetic.

We are interested in modulo-m arithmetic because that is what a computer implements ($m = 2^{32}$ or $m = 2^{64}$).

The set of constraints

$$\begin{aligned} x - y &\in [10, 11] \\ y - z &\in [7, 8] \\ x - z &\in [1, 4] \end{aligned}$$

has no solution in standard arithmetic.

In modulo-16 arithmetic, one solution is

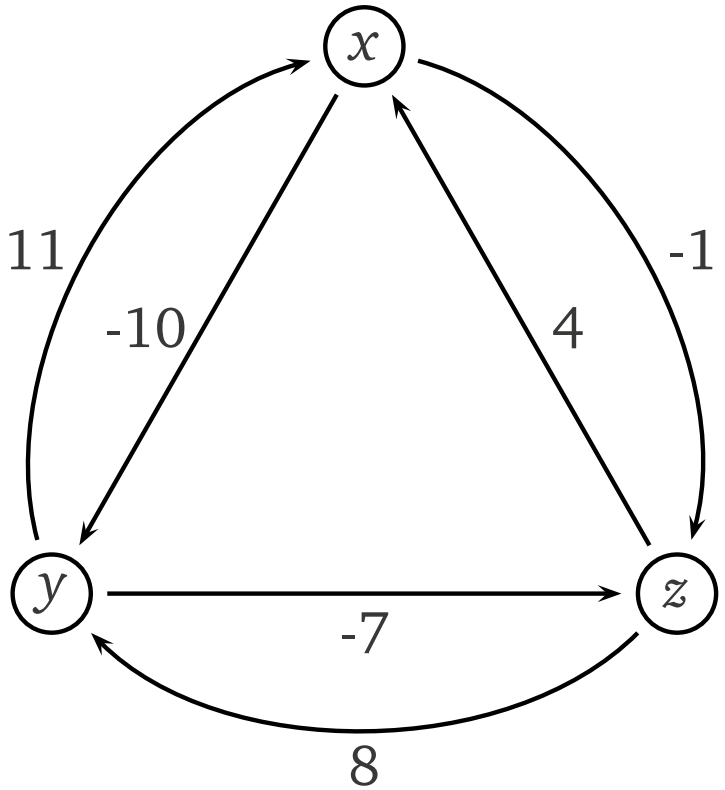
$$\begin{aligned} x &= 11 \\ y &= 0 \\ z &= 8 \end{aligned}$$

Solving Difference Constraints

We have a fast algorithm for DC. It reduces the problem to the "no-negative-cycle" problem for weighted graphs:

The constraints rephrased:

$$\begin{array}{rcr} x - y & \leq & 11 \\ y - x & \leq & -10 \\ y - z & \leq & 8 \\ z - y & \leq & -7 \\ x - z & \leq & 4 \\ z - x & \leq & -1 \end{array}$$



The Bellman-Ford algorithm can solve the latter efficiently.



Solving Modulo m Difference Constraints

Solving Modulo m Difference Constraints

How hard can it be to adapt the DC algorithm so that it works correctly in the context of machine arithmetic, that is, with modulo m arithmetic?

How long should you keep trying when you can't find a good algorithm?

Computational complexity theory helps ...

The Idea of Problem Reduction, Roughly

Consider problems P and Q.

Suppose we can "faithfully" translate, without too much effort, each instance p of P into an instance q of Q.

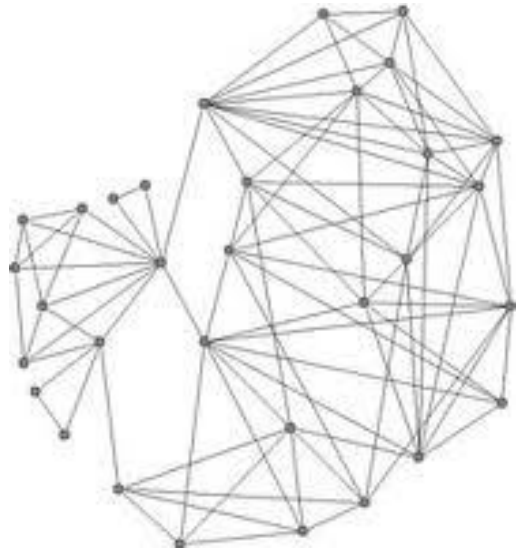
By faithfully we mean that it should be straightforward to extract a solution to p from a solution to q.

So to solve p we just

1. translate it to q,
2. run the algorithm for Q and
3. translate the result back.

If we can find a translation, it means that P is no harder than Q.

Take Two Rather Different Problems ...

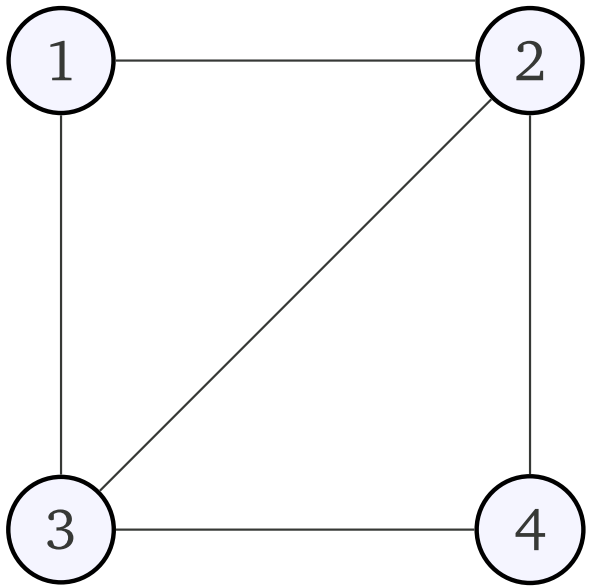


Graph 3-colouring, $3COL$:
An instance is a graph; output is "yes" if and only if the graph can have its nodes coloured using just three colours, in such a way that no two connected nodes are given the same colour.

$$\begin{array}{ccc} x_{11} - x_{12} & \in & [k_1, k_1'] \\ \vdots & & \vdots \\ x_{i1} - x_{i2} & \in & [k_i, k_i'] \\ \vdots & & \vdots \\ x_{n1} - x_{n2} & \in & [k_n, k_n'] \end{array}$$

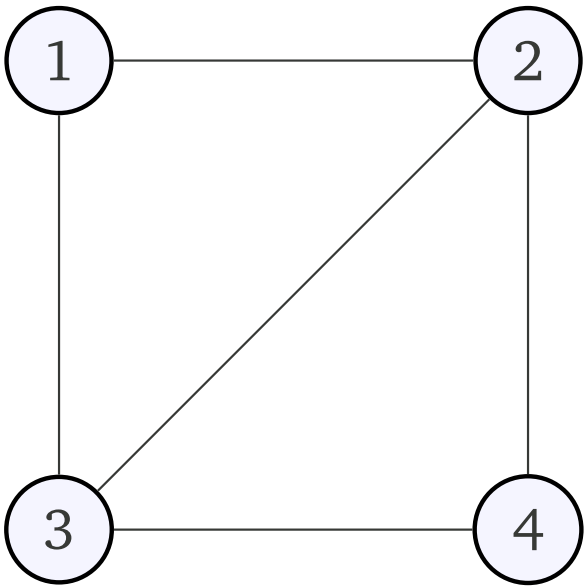
DC_m:
The difference constraint problem modulo m.

Faithfully Translating $3COL$ to DC_m



$\Rightarrow$

Faithfully Translating $3COL$ to DC_m



$\Rightarrow$

$$
\begin{aligned}
x_1 - x &\in [0,2] \\
x_2 - x &\in [0,2] \\
x_3 - x &\in [0,2] \\
x_4 - x &\in [0,2]
\end{aligned}
$$

$$
\begin{aligned}
x_1 - x_2 &\in [1, m-1] \\
x_1 - x_3 &\in [1, m-1] \\
x_2 - x_3 &\in [1, m-1] \\
x_2 - x_4 &\in [1, m-1] \\
x_3 - x_2 &\in [1, m-1]
\end{aligned}
$$

This way the graph can be 3-coloured if and only if the set of constraints has a solution. We have reduced $3COL$ to DC_m.

So What Has that Reduction Achieved?

So What Has that Reduction Achieved?

- We decide not to waste time trying to find an efficient algorithm for DC_m.

So What Has that Reduction Achieved?

- We decide not to waste time trying to find an efficient algorithm for DC_m.
- Because are trying to solve a problem that is at least as hard as graph 3-colouring
- And 3-colouring is known to be hard

A Handle on Relative Hardness: $\leq_P$

Problem Reduction more precisely

To say that the problem translation "should be done without too much effort" will just mean "must be doable in polynomial time."

Definition

A is polynomial-time reducible to B, $A \leq_P B$ iff there is some polynomial-time computable function f, such that for all w,

$$w \in A \text{ iff } f(w) \in B.$$

Theorem: If $A \leq_P B$ and $B \in P$ then $A \in P$.

NP-Completeness

B is NP-hard

iff every $A \in NP$ is reducible to B in polynomial time.

NP-Completeness

B is NP-hard

iff every $A \in NP$ is reducible to B in polynomial time.

B is NP-complete

iff

1. $B \in NP$, and
2. B is NP-hard.

NP-Completeness

B is NP-hard

iff every $A \in NP$ is reducible to B in polynomial time.

B is NP-complete

iff

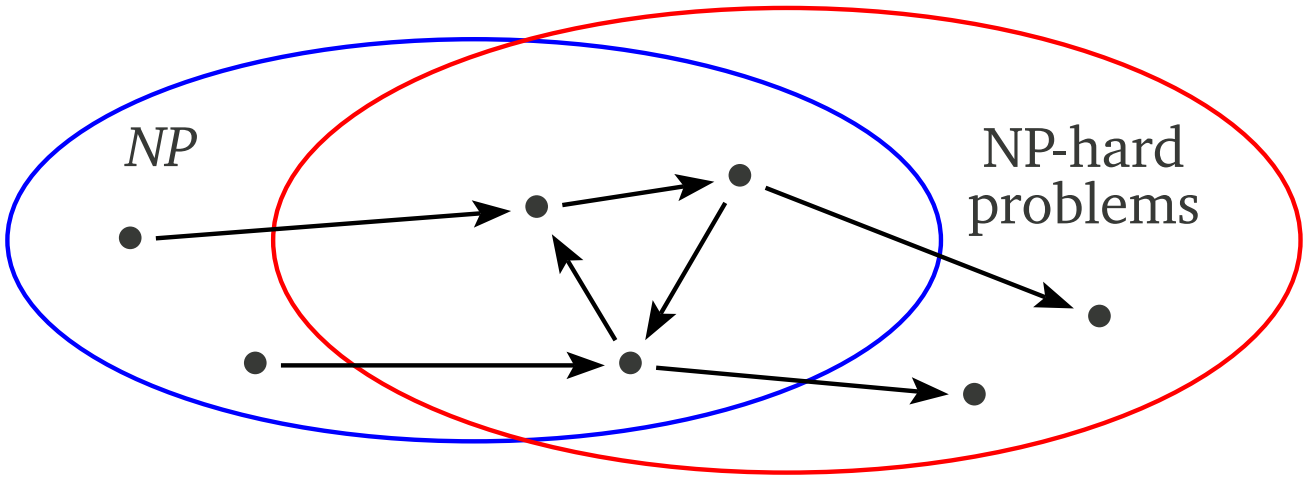
1. $B \in NP$, and
2. B is NP-hard.

NP-complete problems NPC interesting because:

Theorem: If $B \in NPC$ and $B \in P$ then $P = NP$.

NP-Completeness and NP-Hardness

NP-complete problems are the NP-hard problems in *NP*.



Here arrows indicate polynomial time reduction (a transitive relation).

Note: All problems in *NPC* stand or fall together!

NP-Complete Problems

NP-complete problems

- SAT, SUBSET-SUM, 3COL, LPATH, and HAM, ...
- thousands of interesting and practically relevant problems
- We do not know of solutions
  (that are essentially better than exhaustive search)

Complexity theory

- interest in NP-completeness and related concepts
- Many other interesting complexity classes
  (including space complexity classes, probabilistic classes, ...)

Open Problems

So we don't know whether $P = NP$, and there are many other unsolved problems of similar type. We know:

$$P \subseteq NP$$

Dealing with NP-Completeness

Pseudo-polynomial problems

- e.g. SUBSET-SUM and KNAPSACK are in this class
- Unless you have really large data, don't worry; for reasonably-sized sets and numbers, the bad behaviour will not have kicked in yet.

SAT solvers

Clever engineering to push the boundary slowly

Approximation algorithms

Settle for less than perfection

Live happily with intractability

Sometimes the bad instances never turn up in practice

Summary and Next Lecture

Today

- Hard problems
- P vs. NP
- Sometimes we run into more trouble than mere intractability!
- There are practically important computational problems that have no algorithmic solution!

Next

- Exam information